

ME647 - Assignment 1 - 80 Marks

Read the instructions, carefully:

- **Deadline:** Submit by **February 4, 2026, 2330 hrs.** Late submissions upto 1 day will be graded out of 60%, and later than 1 day will be graded 0.
 - **Submission Format:** Your solutions should be 1 zip file, called “FullName-RollNumber.zip” on HelloIITK. This should contain 1 PDF “FullName-RollNumber.pdf” with all the solutions and figures (make sure labels and fonts on figures are readable in the PDF). Also submit your Python/Matlab scripts - combine all the functions and codes to one file “FullName-RollNumber.py”, which has different sections, and not multiple code files.
 - **AI:** If you make use of AI assistance for the codes, you need to *disclose it clearly*. Remember, the solutions you get may not be correct. Moreover, you **must be able to explain** and stand by your submitted solutions and codes. **Blindly copying from AI will fetch 0 marks.**
 - Bonus for answers formatted in LaTeX (if correct).
 - While discussing with your classmates is encouraged, there will be **zero tolerance for plagiarism, copying from each other will fetch 0 marks.**
-

1 Fluid Flow Equations

The incompressible Navier-Stokes equations can be written for the i -th velocity component as

$$\partial_t u_i + u_j \partial_j u_i = -\partial_i(p/\rho) + \nu \partial_j \partial_j u_i \quad (1)$$

along with the incompressibility constraint $\partial_i u_i = 0$. We will refer to Eq.1 as (NSE) $_i$.

1. Analogously to the kinetic energy, we often encounter *enstrophy* which is given as $\omega_i \omega_i / 2$. Take the curl of the Navier-Stokes equation by multiplying it with $\epsilon_{pqi} \partial_q (NSE)_i$ to obtain the transport equation of vorticity ω_p . Then multiply with ω_p and obtain an equation (somewhat) similar to the kinetic energy equation. Compare the terms with the kinetic energy equation. Show all the steps and present a clean version of the derivation. (15 Marks)

2 Turbulence Statistics

Attached you will find a two-dimensional cut from a direct numerical simulation of three-dimensional homogeneous, isotropic turbulence from the Johns Hopkins Turbulence Database. The details of your dataset are as follows, along with dimensionless parameters:

- Grid Size: $N_x \times N_y = 1024 \times 1024$ grid cells (which is a 2D plane, at $z = 0$, from a 1024^3 simulation). The lateral size $L_x = L_y = 2\pi$ [au], and the length of each grid cell is $dx = L/N_x$. The kinematic viscosity $\nu = 0.000185$ [au], and an integral lengthscale of $\mathcal{L} = 1.36$ [au].
- Velocity fields in dataset: u, v, w or u_i with $i \in \{1, 2, 3\}$

- Boundaries: Periodic (i.e. $u_i(l, y) = u_i(l + 2\pi, y)$)

You will also find a simple Python code to load this data. Repeated indices, like $u_i u_i$, imply summation in Einstein notation, angle brackets $\langle \cdot \cdot \rangle$ denote spatial-averaging over all points.

1. Find the Reynolds Number and Kolmogorov Scales η , τ_η and u_η . For the large scales, use $\mathcal{L}$ as lengthscale and the root-mean-square velocity $\langle u_i(\mathbf{x}) u_i(\mathbf{x}) \rangle^{1/2}$ as a velocity scale U . What is the grid cell size with respect to the Kolmogorov length? (10 Marks)
2. Plot the two-dimensional normalized kinetic energy field $\tilde{k} = k/\langle k \rangle$ where $k(\mathbf{x}) = u_i(\mathbf{x}) u_i(\mathbf{x})/2$. You can plot over a smaller subsection (512×512) to more clearly see the features. Suitably adjust the colorbar. (5 Marks)
3. First expand $S_{ij} S_{ij}$ by hand to familiarize yourself with all the terms, where $S_{ij} = (\partial u_i / \partial x_j + \partial u_j / \partial x_i)/2$. Plot the two-dimensional normalized dissipation field $\tilde{\epsilon} = \epsilon/\langle \epsilon \rangle$ where $\epsilon(\mathbf{x}) = 2\nu S_{ij}(\mathbf{x}) S_{ij}(\mathbf{x})$. You can calculate all gradients like $\partial u / \partial x$ etc using central-differencing. (10 Marks)
4. Calculate and plot the vorticity. Since you have planar data, you can compute only one vorticity component with the in-plane velocity cross gradients. Hence calculate $\omega_z(\mathbf{x})$. Use a second-order central differencing scheme for approximating the gradients required. Plot the planar vorticity field, again over a subsection, ω_z normalized by its root-mean-square value $\omega_{\text{rms}} = \langle \omega_z^2(\mathbf{x}) \rangle^{1/2}$, with a colorbar. (5 Marks)
5. Using the field data, calculate the centered **PDF** (normalized histogram), and report the **Skewness** and **Kurtosis** of the quantities below. Add a normal (Gaussian) distribution with mean 0 and standard deviation of 1 i.e. $\mathcal{N}(0, 1)$ as a guide-to-eye to compare with the various quantities in each figure, and therefore normalize your pdf with the standard deviation of the data (x -axis should be x/σ while y -axis should be $p(x) \times \sigma$). *Make sure to plot these on a logarithmic y-axis.* (20 Marks)
 - (a) The velocity components— u, v, w —in the same figure. Use different symbols to show them. What sort of a distribution is this?
 - (b) The vorticity component ω_z . What do you find about the flatness of this quantity?
 - (c) The velocity gradients $\partial u / \partial x$ and $\partial v / \partial y$. What do you find about the skewness of these quantities.
 - (d) The enstrophy ω_z^2 .

Important: In your dataset, any velocity component, say u will be of the form $u[x, y]$, therefore the axis 0,1 (1,2 in Matlab) correspond to x, y directions. So, $\partial u / \partial x$ at the point (i, j) will be:

$\frac{du}{dx}[i, j] = (u[i+1, j] - u[i-1, j]) / (2 * dx)$
--

6. Let us consider the u velocity field. Use this to calculate and plot an approximate non-linear term $N_{\text{lin}} = u_j \partial_j u \approx u \partial_x u + v \partial_y u$ and an approximate viscous term $\text{Visc} = \nu \partial_j \partial_j u = \nu (\partial_x^2 + \partial_y^2) u$. You can now plot the local Reynolds number field $Re = \frac{|N_{\text{lin}}|}{|\text{Visc}|}$. How does this compare to the kinetic energy and dissipation fields above spatially? You can quantify this with the correlation-coefficient $\rho(k, Re)$, $\rho(\epsilon, Re)$ and $\rho(k, \epsilon)$. Summarize your observations about the spatial distribution of these fields in the light of these results. (15 Marks)

Hints and Suggestions

- You may use the following syntax to plot the 2D fields:

```
1 skip = 5
2 plt.pcolor(ke[:,::skip,::skip], vmin=0, vmax=10)
```

The above code will plot every 5th point of the dataset in the x and y directions, hence reducing memory usage. You can use this syntax to check a good range for $vmin$ and $vmax$, which shows the field clearly without clipping it at low or high values, and structures are visible. For fields with negative values, use $vmin=-val$ and $vmax=+val$, for some value of val , to have equal range on both sides. For fields whose PDF is highly non-Gaussian (like vorticity and dissipation), you should not pick $vmax$ to be close to the maximum value of the field, this will hide everything else. Pick $vmax$ to be maybe 3-4 times the root-mean-square value instead.

You can finally change to $skip=1$, to plot the full field, when you are satisfied with your plot.

- For PDFs, you should first create a histogram of the data, for which you will need to send in a 1D array to the histogram function, along with the number of bins. You can use, for example

```
1 hist, bins = np.histogram(field.ravel(), bins=100)
```

to return a histogram of the field over 100 bins (the function actually returns the bin-edges, hence “bins” is 1 longer than the length of “hist”). It is best to send in data after subtracting the mean, so that it is already “centered”. You will have to convert this histogram to a PDF with a normalization. The CDF is simply a cumulative integral that will run over the bin range you have created, which you can do with either a loop or with a python list

```
1 cdf = [ np.sum(pdf[:i]) for i in range(len(hist)) ]
```

- I highly recommend that you save the calculated PDFs and CDFs in small text files, and use a different script to plot the data. Typically plotting takes several iterations to optimize, this will save you time as you don’t need to do the longer PDF calculation each time, and can simply read a small text file and plot it. You can use

```
1 np.savetxt(...)
2 np.loadtxt(...)
```

- You can save figures using

```
1 plt.savefig("Figure1.png", bbox_inches='tight', dpi=240)
```

- Calculate Skewness and Kurtosis from the data field directly, and not as moments of the PDF. Find the relevant formulas for doing so.
- For central differencing, you may want to explore using the following function for faster calculations, rather than two nested loops over x and y , since the data is periodic.

```
1 np.roll( ... )
```

Of course, do not forget to divide the ‘differences’ with the right length, to approximate $\partial u / \partial x \approx \Delta u / \Delta x$, where Δx will be measured in terms of the grid cell length.

- The Laplacian $(\partial_x \partial_x + \partial_y \partial_y)u$ can be calculated by applying a [five point stencil](#).

All the best!